

Problem

Submissions

Leaderboard

Discussions

Profile

When a subclass inherits from a superclass, it also inherits its methods; however, it can also override the superclass methods (as well as declare and implement new ones). Consider the following Sports class:

```
class Sports{
    String getName(){
        return "Generic Sports";
    }
    void getNumberOfTeamMembers(){
        System.out.println( "Each team has n players in " + getName() );
    }
}
```

Next, we create a Soccer class that inherits from the Sports class. We can override the getName method and return a different, subclass-specific string:

```
class Soccer extends Sports{
    @Override
    String getName(){
        return "Soccer Class";
    }
}
```

Note: When overriding a method, you should precede it with the `@Override` annotation. The parameter(s) and return type of an overridden method must be exactly the same as those of the method inherited from the supertype.

Task

Complete the code in your editor by writing an overridden `getNumberOfTeamMembers` method that prints the same statement as the superclass' `getNumberOfTeamMembers` method, except that it replaces `n` with `11` (the number of players on a Soccer team).

Output Format

When executed, your completed code should print the following:

```
Each team has 11 players in Soccer Class
```

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

Line: 18 Col: 1

Upload Code as File

Test against custom input

Run Code

Submit Code

You have earned 10.00 points!

You are now 92 points away from the gold level for your java badge.

8%

158/250

Congratulations

You solved this challenge. Would you like to challenge your friends?

Next Challenge

When a method in a subclass overrides a method in superclass, it is still possible to call the overridden method using **super** keyword. If you write `super.func()` to call the function `func()`, it will call the method that was defined in the superclass.

You are given a partially completed code in the editor. Modify the code so that the code prints the following text:

```
Hello I am a motorcycle, I am a cycle with an engine.  
My ancestor is a cycle who is a vehicle with pedals.
```

```
23 }  
24 class Solution{  
25     public static void main(String []args){  
26         Motorcycle M=new Motorcycle();  
27     }  
28 }  
29
```

Line: 17 Col: 1

Upload Code as File

☐ Test against custom input

Run Code

Submit Code



You have earned 10.00 points!

You are now 82 points away from the gold level for your java badge.

18%

168/250

Congratulations

You solved this challenge. Would you like to challenge your friends?



Next Challenge

Test case 0

Compiler Message

Problem

"The singleton pattern is a design pattern that restricts the instantiation of a class to one object. This is useful when exactly one object is needed to coordinate actions across the system."

[- Wikipedia: Singleton Pattern](#)

Complete the Singleton class in your editor which contains the following components:

1. A private Singleton non parameterized constructor;
2. A public String instance variable named `str`;
3. Write a static method named `getSingletonInstance` that returns the single instance of the Singleton class.

Once submitted, our hidden Solution class will check your code by taking a String as input and then using your Singleton class to print a line.

Input Format

You will not be handling any input in this challenge.

Output Format

You will not be producing any output in this challenge.

Sample Input

hello world

Sample Output

Hello I am a singleton! Let me say hello world to you

```

18     if (instance == null) {
19         instance = new Singleton();
20     }
21     return instance;
22 }
23 }
24
25
26 public class Main {
27
28     public static void main(String args[]) throws Exception {
29
30         Scanner sc = new Scanner(System.in);
31         Singleton s1 = Singleton.getSingletonInstance(); //retrive the single instance
32         Singleton s2 = Singleton.getSingletonInstance();
33         assert(s1==s2);
34
35         //verify that the constructor is private
36     }
37 }

```

Line: 1 Col: 1

Upload Code as File

☐ Test against custom input

Run Code

Submit Code



You have earned 15.00 points!

You are now 67 points away from the gold level for your java badge.

33%

183/250

Congratulations

You solved this challenge. Would you like to challenge your friends?

Next Challenge

Problem

Missions

derboard

- ## Discussions

Line: 211 Col: 6

☐ Test against custom input

Submit Code



73% 223/250

Next Challenge

You solved this challenge. Would you like to challenge your friends?

Compiler Message

Success

Input (stdin)

[illegible]

2 4 7 2 5 1

0 1 0 0 1

12

13

34

Download

Java Iterator class can help you to iterate through every element in a collection. Here is a simple example:

```
import java.util.*;

public class Example{

    public static void main(String []args){
        ArrayList mylist = new ArrayList();
        mylist.add("Hello");
        mylist.add("Java");
        mylist.add("4");
        Iterator it = mylist.iterator();
        while(it.hasNext()){
            Object element = it.next();
            System.out.println(String.valueOf(element));
        }
    }
}
```

In this problem you need to complete a method func. The method takes an ArrayList as input. In that ArrayList there is one or more integer numbers, then there is a special string "###", after that there are one or more strings. A sample ArrayList may look like this:

```
element[0]>=>42
element[1]>=>10
element[2]>=>"###"
element[3]>=>"Hello"
element[4]>=>"Java"
```

You have to modify the func method by editing at most 2 lines so that the code only prints the elements after the special string "###". For the sample above the output will be:

```
Hello
Java
```

Note: The stdin doesn't contain the string "###", it is added in the main method.
To restore the original code in the editor, click the top left icon on the editor and create a new buffer.

```
16 @SuppressWarnings({ "unchecked" })
17 public static void main(String []args){
18     ArrayList mylist = new ArrayList();
19     Scanner sc = new Scanner(System.in);
20     int n = sc.nextInt();
21     int m = sc.nextInt();
22     for(int i = 0;i<n;i++){
23         mylist.add(sc.nextInt());
24     }
25
26     mylist.add("###");
27     for(int i=0;i<m;i++){
28         mylist.add(sc.next());
29     }
30
31     Iterator it=func(mylist);
32     while(it.hasNext()){
33         Object element = it.next();
34         System.out.println((String)element);
35     }
36 }
```

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

 You have earned 15,000 points!
You are now 12 points away from the gold level for your java badge.

88%

238/250

Congratulations

You solved this challenge. Would you like to challenge your friends?

Next Challenge

A Java Interface can only contain method signatures and fields. The interface can be used to achieve polymorphism. In this problem, you will practice your knowledge on interfaces.

You are given an interface `AdvancedArithmetic` which contains a method signature `int divisorSum(int n)`. You need to write a class called `MyCalculator` which implements the interface.

`divisorSum` function just takes an integer as input and return the sum of all its divisors. For example divisors of 6 are 1, 2, 3 and 6, so `divisorSum` should return 12. The value of `n` will be at most 1000.

Read the partially completed code in the editor and complete it. You just need to write the `MyCalculator` class only. Your class shouldn't be public.

Sample Input

6

Sample Output

```

I Implemented: AdvancedArithmetic
12
    
```

Explanation

Divisors of 6 are 1,2,3 and 6. $1+2+3+6=12$.

```

23     MyCalculator my_calculator = new MyCalculator();
24     System.out.print("I implemented: ");
25     ImplementedInterfaceNames(my_calculator);
26     Scanner sc = new Scanner(System.in);
27     int n = sc.nextInt();
28     System.out.print(my_calculator.divisorSum(n) + "\n");
29     sc.close();
30 }
31
32 /*
33  * ImplementedInterfaceNames method takes an object and prints the name of the interfaces it implements
34  */
35 static void ImplementedInterfaceNames(Object o){
36     Class[] theInterfaces = o.getClass().getInterfaces();
37     for (int i = 0; i < theInterfaces.length; i++){
38         String interfaceName = theInterfaces[i].getName();
39         System.out.println(interfaceName);
40     }
41 }
    
```

Line: 0 Col: 1

☐ Test against custom input



You have earned 10.00 points!

You are now 2 points away from the gold level for your java badge.

98% 248/250

Congratulations

You solved this challenge. Would you like to challenge your friends? [Twitter](#) [LinkedIn](#)

Test case 0

Problem

Submissions

Leaderboard

Discussions

Editorial

According to Wikipedia, a factory is simply an object that returns another object from some other method call, which is assumed to be "new".

In this problem, you are given an interface Food. There are two classes Pizza and Cake which implement the Food interface, and they both contain a method getType().

The main function in the Main class creates an instance of the FoodFactory class. The FoodFactory class contains a method getFood(String) that returns a new instance of Pizza or Cake according to its parameter.

You are given the partially completed code in the editor. Please complete the FoodFactory class.

Sample Input 1

cake

Sample Output 1

The factory returned class Cake
Someone ordered a Dessert!

Sample Input 2

pizza

Sample Output 2

The factory returned class Pizza
Someone ordered Fast Food!

```
63
64
65     public static void forbidExit() {
66         final SecurityManager securityManager = new SecurityManager() {
67             @Override
68             public void checkPermission(Permission permission) {
69                 if (permission.getName().contains("exitVM")) {
70                     throw new ExitTrappedException();
71                 }
72             }
73         };
74         System.setSecurityManager(securityManager);
75     }
76 }
77
78
79
80
81
```

Line: 23 Col: 35

Upload Code as File

☐ Test against custom input

Run Code

Submit Code



You have earned 15.00 points!
9/94 challenges solved.



Congratulations

You solved this challenge. Would you like to challenge your friends? [Twitter](#) [LinkedIn](#)

Next Challenge

You are required to compute the power of a number by implementing a calculator. Create a class MyCalculator which consists of a single method `long power(int, int)`. This method takes two integers, `n` and `p`, as parameters and finds n^p . If either `n` or `p` is negative, then the method must throw an exception which says "**n or p should not be negative**". Also, if both `n` and `p` are zero, then the method must throw an exception which says "**n and p should not be zero.**"

For example, -4 and -5 would result in `java.lang.Exception: n or p should not be negative`.

Complete the function `power` in class `MyCalculator` and return the appropriate result after the power operation or an appropriate exception as detailed above.

Input Format

Each line of the input contains two integers, `n` and `p`. The locked stub code in the editor reads the input and sends the values to the method as parameters.

Constraints

- $-10 \leq n \leq 10$
- $-10 \leq p \leq 10$

Output Format

Each line of the output contains the result n^p , if both `n` and `p` are positive. If either `n` or `p` is negative, the output contains "**n and p should be non-negative**". If both `n` and `p` are zero, the output contains "**n and p should not be zero.**" This is printed by the locked stub code in the editor.

Sample Input 0

```
3 5
2 4
0 0
-1 -2
-1 3
```

Sample Output 0

```
243
16
```

```
23
24 while (in.hasNextInt()) {
25     int n = in.nextInt();
26     int p = in.nextInt();
27
28     try {
29         System.out.println(my_calculator.power(n, p));
30     } catch (Exception e) {
31         System.out.println(e);
32     }
33 }
34 }
```

Line: 2 Col: 1

Upload Code as File

☐ Test against custom input

Run Code

Submit Code



You have earned 15,000 points!
0/64 challengers solved.

14%

Congratulations

You solved this challenge. Would you like to challenge your friends? [Twitter](#) [LinkedIn](#)

Next Challenge

Test case 0

Compiler Message

Test case 1

Success

Problem

Submissions

Leaderboard

Discussions

Editorial

Exception handling is the process of responding to the occurrence, during computation, of exceptions — anomalous or exceptional conditions requiring special processing — often changing the normal flow of program execution. (Wikipedia)

Java has built-in mechanism to handle exceptions. Using the try statement we can test a block of code for errors. The catch block contains the code that says what to do if exception occurs.

This problem will test your knowledge on try-catch block.

You will be given two integers x and y as input, you have to compute x/y . If x and y are not **32** bit signed integers or if y is zero, exception will occur and you have to report it. Read sample Input/Output to know what to report in case of exceptions.

Sample Input 0:

```
10
3
```

Sample Output 0:

```
3
```

Sample Input 1:

```
10
Hello
```

Sample Output 1:

```
java.util.InputMismatchException
```

Sample Input 2:

```
10
0
```

```
10
10
20
```

```
}
}
```

line: 20 col: 1

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

4
pts

java

You have earned 10.00 points!

9/64 challenges solved.

14%

Congratulations

You solved this challenge. Would you like to challenge your friends?



Next Challenge

Input Format

In the first line, there will be an integer T denoting number of pairs. Each of the next T lines will contain two strings separated by a single space.

Constraints:

- $1 \leq T \leq 100000$
- Length of each string is atmost 5 and will consist lower case letters only.

Output Format

Print T lines. In the i th line, print number of unique pairs you have after taking i th pair as input.

Sample Input

```
5
john tom
john mary
john tom
mary anna
mary anna
```

Sample Output

```
1
2
2
3
3
```

Explanation

- After taking the first input, you have only one pair: (john,tom)
- After taking the second input, you have two pairs: (john, tom) and (john, mary)
- After taking the third input, you still have two unique pairs.
- After taking the fourth input, you have three unique pairs: (john,tom), (john, mary) and (mary, anna)
- After taking the fifth input, you still have three unique pairs.

```
15     for (int i = 0; i < t; i++) {
16         pair_left[i] = s.next();
17         pair_right[i] = s.next();
18     }
19
20     Set<List<String>> pairs = new HashSet<List<String>>();
21     for (int i = 0; i < t; i++) {
22         List<String> pair = Arrays.asList(pair_left[i], pair_right[i]);
23
24         pairs.add(pair);
25
26         System.out.println(pairs.size());
27     }
28 }
29
30 }
```

Line: 27 Col: 30

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

Download

```
1 5
2 john tom
3 john mary
4 john tom
```